Optimizing Hypothetical Nuclear Excavation for a Qattara Canal: A Mixed-Integer Linear Programming Study

William Michelsen (wimi@itu.dk) Maciej Jałocha (macja@itu.dk)
(Equal contributions)

11.04.2026

1 Introduction

If the Qattara depression was filled with water from the Meditteran sea it could positively affect the climate in the northern Africa. However, it would require building a channel of ¿75 kilometers. To eradicate such masses of dirt efficiently, nuclear bombs' blasts have been proposed.

The goal of this report is to find insights on how to effectively place the bombs. Using Mixed Integer Linear programming, we studied how different placements and types of bombs affect the resulting channel shape, and how many bombs it would require.

2 Data

The data used in the study are 100 spatial points sampled on the hypothetical channel path. They contains a latitude, longitude and amount of the height (in meters) above the sea level.



Figure 1

Figure 2: Sampled 100 coordinates of the proposed Channel Path overlaid on a satellite map

3 Methods

First, we transform the data from into distances from the sea, then interpolate more points, and then we study the bombs placement using Mixed Integer Linear Programming

3.1 Spatial Coordinates to Distances

The desired data for our study is composed of pairs (distance, height), but now we have (latitude, longitude, height). We convert latitude and longitude of each point to the distance. To do so we use the haversine formula to compute distances from the coordinates [1]. That is, given two points

(ϕ_1, λ_1) and (ϕ_2, λ_2), where latitude and longitude are first converted from degrees to radians, we compute the central angle between them using the haversine formula. Let

$$\mathrm{hav}(x) = \frac{1 - \cos(x)}{2}.$$

Then

$$\mathrm{hav}(\theta) = \mathrm{hav}(\phi_2 - \phi_1) + \cos(\phi_1)\cos(\phi_2)\,\mathrm{hav}(\lambda_2 - \lambda_1),$$

so that

$$\theta = 2\arcsin\left(\sqrt{\mathrm{hav}(\theta)}\right), \qquad d = R\theta,$$

where R is the Earth's mean radius (6371km) and d is the distance between the two locations.

3.2 Interpolation

To obtain a better bombs placement, we obtain a finer representation of the terrain.
We use piecewise linear interpolation between consecutive sampled points. For each pair of neighboring observations, we compute the slope as the change in height divided by the change in distance, and then use that slope to estimate heights at intermediate locations. In this way, the original data is transformed onto a uniform grid with one point every 250 m.

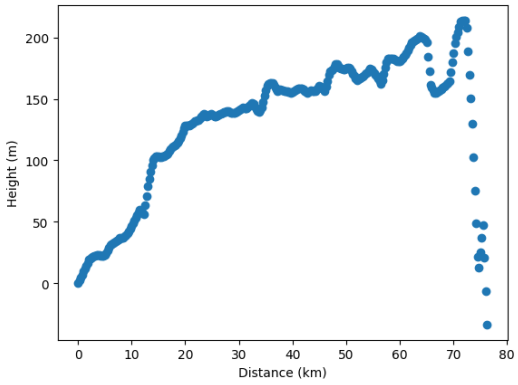


Figure 3: Scatter plot of the original and interpolated terrain heights, along the proposed channel path, sampled every 250 m.

3.3 Mixed Integer Linear Programming

Linear programming is a mathematical model used to find the best possible outcome, like minimizing errors, while respecting a set of linear constraints. Standard linear programming assumes variables are continuous. However, because our problem requires a strict yes-or-no decision for every location (whether to place a bomb or not), we use mixed integer linear programming. These binary decisions represent our decision variables, which in linear programming, are the unknown values we are trying to find [2]. Integer linear programming forces these decision variables to be integers which fits our problem.

To compute the optimal placements from the vast number of possible combinations, we utilized the CBC [3] and GLPK [4], which are standard algorithms designed specifically for mixed integer problems.

Mathematically, we define a binary variable $x_i \in \{0, 1\}$ for each point i, where $x_i = 1$ if a bomb is place there and 0 otherwise. Based on the blast parameters (K_0, K_1, K_2) the total amount of dirt removed at point i, denoted as R_i, is the sum of the yields from a bomb at i and any neighboring bombs. We can denote it like this:

$$R_i = K_2 x_{i-2} + K_1 x_{i-1} + K_0 x_i + K_1 x_{i+1} + K_2 x_{i+2}$$

As said, a typical component of linear programming problems is respecting some constraints. Our situation is no different. To ensure the channel functions, the removed dirt must meet the required channel depth (CHD) compared to the terrain height (H_i):

$$R_i \geq H_i + CHD \text{for all } i$$

Instead of simply solving an obvious minimization problem such as minimizing the total number of bombs, we optimize for the overall smoothness of the channel. Another constraint is also introduced, preventing any bombs from being placed at neighboring grid point.

3.3.1 Handling absolute value objective

Optimizing for a smooth channel means we want the final depth to be aa close to the target depth as possible, avoiding any unnecessarily deep craters. This requires minimizing the absolute difference between the actual depth and the target depth.

Since linear programming solvers cannot process non-linear values directly, we linearized the objective function. At each location i, we introduced a variable $s_i > 0$ to represent the deviation. We bounded s_i with two constraints to cover both positive and negative deviations: $s_i \geq R_i - H_i - CHD$ and $s_i \geq -(R_i - H_i - CHD)$.

4 Experiments

Specifically, we obtain the results on:

- What is the bombs placement, when optimising for minimum amount of nukes versus when optimising for the channel smoothness.
- How does bombs placement change when no bomb can be placed within blast radius of other bombs. Smoothness objective is used.
- How does bombs placement change when allowing for bombs with different yield?

4.1 Details

We study a setup where bombs have a blast radius of approximately 500 m, i.e. within two closest points. The farther a point is (within a radius), the less dirt is removed, e.g. a bomb removes 300 m at its position, 140 m one point away and 40 m two points away. To note, the dirt deposited back into the channel is not included in computations.

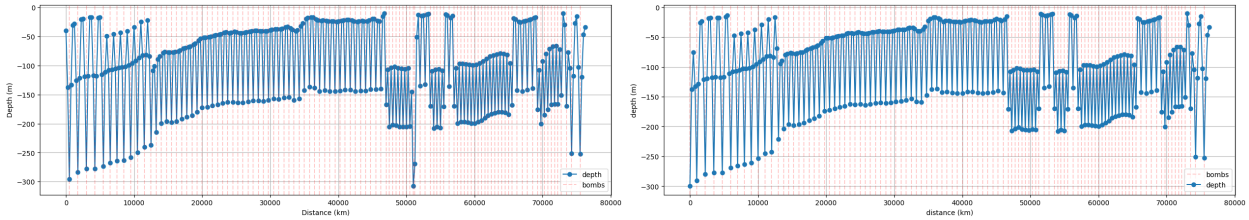
As channel smoothness we meant optimising for the $min \sum_{i=1}^{n} R_i - H_i - CHD$ where R_i is obtained channel depth, H_i is the height above sea level, given by the data, and CHD is a minimum channel depth, set to 10 m. Note that the whole term cannot be negative.

We assume an initial blast radius of (300, 140, 40). FOr dial-a-yield experiment we test the setups in 1

5 Results

5.1 Optimising for amount of nukes versus for the channel smoothness

The experiments below assumed $(K0, K1, K2) = (300, 140, 40)$



(a) For: the minimum number of bombs.

(b) For: channel smoothness.

Figure 4: Comparison of the resulting channel depth profiles under the two optimization objectives. Y axis represents depth (m) and X axis represents distance (km). The connected scatter show the channel profile. The vertical lines mark bombs positions.

Bombs in 4 seem almost evenly distributed. The channel depth profiles seem similar. Moreover, both objectives end up using 108 bombs.

5.2 No adjacent bombs constraint

The solution with a constraint that no bomb can be placed within the full blast radius of another bomb is infeasible. Therefore, we relaxed the requirement and instead imposed that no two bombs may be placed at adjacent grid points.
Under this weaker restriction, the smoothness-based model remains feasible and produces the solution shown in Figure 5. Compared with the unconstrained smoothness solution, the placements are almost the same. The number of bombs is also the same.

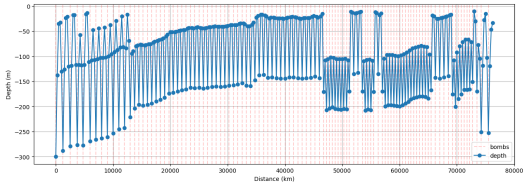
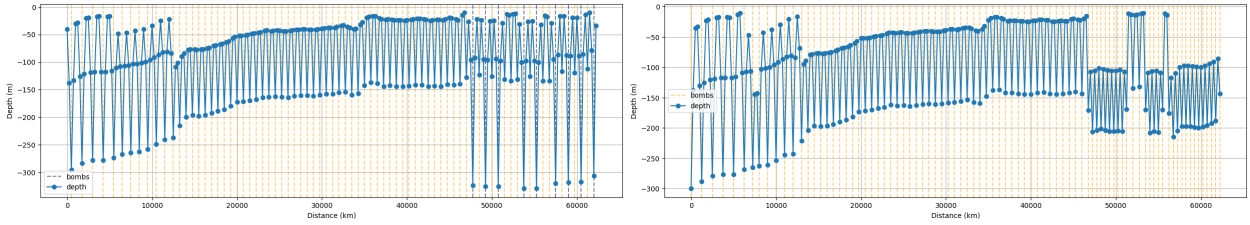


Figure 5: Result of the smoothness-based optimization with the additional constraint that no two bombs are placed at adjacent points.

5.3 Dial a yield possibility

We assume smoothness objective and no adjacent bombs constraints.

We were unable to obtain a solution for all points in the channel under 19 minutes, running on a computer with M3. Therefore we truncate the data to 250 datapoints.



(a) Dial-a-yield (settings 0, 1, 2) solution on the truncated instance.

(b) Baseline using only setting 1 on the truncated instance.

Figure 6: Comparison of the truncated-instance solutions with and without the dial-a-yield option. Y axis represents depth (m) and X axis represents distance (km). The connected scatter show the channel profile. The vertical lines mark bombs positions.

Dial-a-yield possibility reduced the amount of bombs from 87 to 78. However, its depth profile does not clearly seem smoother. While smoothness objective takes 55200 instead of 57060 for the baseline, it has 9 extra sudden depth spikes.

In addition, we are seeing that setting 3 is not utilised. Removing that options from the program saves up 15 s (42 s vs 27 s)

6 Discussion

Our experiemtns showed that optimizing for a smooth channel did not require more bombs than simply trying to minimize the total number of nukes. Both objectives resulted in exactly 108 bombs being placed, and their profiles were remarkably similar. This indicates that we can successfully optimize for a good water flow without having to worry about the cost of adding nukes.

Furthermore, when we added the constraint to avoid placing bombs at adjacent points, the model still find a highly effective solution. The placements and the total number of bombs remained almost identical to the unconstrained smoothness solution. Therefore, we can prevent the risk of igniting adjacent bombs without needing to add more bombs and without sacrificng the smoothness of the channel.

Lastly, a dial-a-yield possibility was explored to see if different explosive yields would result in a smoother channel or allow us to use fewer bombs. On a truncated dataset, the dial-a-yield option successfully reduced the number of bombs (from 87 to 78) and improved the mathematical objective score. While the solver minimized the overall mathematical deviation, it achieved this by keeping most of the channel exactly at the target depth at the cost of creating some depth spikes. Therefore, while technically being mathematically smoother, the resulting channel is seemingly practically rougher. Given this decreased practical channel quality and the increase in computation time, utilizing different yields does not seem to be worth the added complexity.

7 Conclusions

The main result of our analysis is that mixed integer linear programming is an effective way to plan the excavation of the Qattara Depression. We demonstrated that we can optimize bomb placements to create a smooth, realistic channel depth. This could be done without increasing the total number of bombs compared to a simple minimization strategy. Some practical safety constraints, like placing bombs apart to prevent them destroying each other, can be made part of the plan without negatively affecting the final channel shape or requiring more explosives. Lastly, our findings suggest that using bombs with varying yields seems to offer little practical benefit. This is because it creates a rougher channel, and the computation required for the planning increases substantially.

References

[1] R. W. Sinnott, *Virtues of the Haversine*, Sky and Telescope, vol. 68, no. 2, p. 159, 1984.

[2] NVIDIA, *Glossary: Mixed-Integer Programming*, https://www.nvidia.com/en-us/glossary/mixed-integer-programming/

[3] COIN-OR Foundation, *Introduction to Cbc*, https://coin-or.github.io/Cbc/intro.html

[4] GNU Project, *GLPK 5.0 Release*, https://lists.gnu.org/archive/html/info-gnu/2020-12/msg00007.html

A Appendix

A.1 Figures

Setting	k_0	k_1	k_2
1	300	140	40
2	500	230	60
3	1000	400	70

Table 1: Dial-a-yield configurations used in the experiments.

A.2 Use of Generative AI

The analysis code was written almost entirely by us, with GitHub Copilot used for debugging. The report was written by us; we used OpenAI's Prism for proofreading, formatting, and figure creation.

A.3 Analysis code

```
# %%
# load ../data/channel_data.txt as csv lat, long, height

import pandas as pd

df = pd.read_csv(
    "../data/channel_data.txt",
    header=None,
    names=["lat", "long", "height"],
    sep="\t",
)
init_lat, init_long, init_height = df.values[0]

# add init points on top of df


# %%
lat1 = 91
long1 = 40
lat2 = 1
```

```
long2 = 40
vector_cosine = (lat1 * lat2 + long1 * long2) / (
    np.sqrt(lat1**2 + long1**2) * np.sqrt(lat2**2 + long2**2)
)
np.arccos(vector_cosine)

# %%
lat1 = np.radians(lat1)
long1 = np.radians(long1)
lat2 = np.radians(lat2)
long2 = np.radians(long2)

phi = lat2 - lat1
lam = long2 - long1
hav_theta = hav(phi) + np.cos(lat1) * np.cos(lat2) * hav(lam)
theta = 2 * np.arcsin(np.sqrt(hav_theta))
theta

# %%
import numpy as np


def hav(angle):
    return (1 - np.cos(angle)) / 2


def get_distance(lat1, long1, lat2, long2):
    R = 6371
    lat1 = np.radians(lat1)
    long1 = np.radians(long1)
    lat2 = np.radians(lat2)
    long2 = np.radians(long2)

    phi = lat2 - lat1
    lam = long2 - long1
    hav_theta = hav(phi) + np.cos(lat1) * np.cos(lat2) * hav(lam)
    theta = 2 * np.arcsin(np.sqrt(hav_theta))

    distance = R * theta
    return distance * 1000

# %% [markdown]
# SHould i do it somehow differently, using haversine formula or whatever? I don't think so?

# %%
# apply along axis
df["distance"] = df.apply(
    lambda row: get_distance(init_lat, init_long, row["lat"], row["long"]), axis=1
)
df

# %%
import matplotlib.pyplot as plt


plt.figure()

plt.scatter(df["distance"], df["height"])

```

```
plt.show()

# %%
df["angle"] = (df["height"].shift(-1) - df["height"]) / (
    df["distance"].shift(-1) - df["distance"]
)

max_distance = df["distance"].max()
interpolated_heights = []
distances = []

curr_distance = 0
last_ref_distance_index = 0
while curr_distance < max_distance:
    distances.append(curr_distance)
    if curr_distance >= df.iloc[last_ref_distance_index + 1]["distance"]:
        last_ref_distance_index += 1

    val = df.iloc[last_ref_distance_index]["height"] + df.iloc[last_ref_distance_index][
        "angle"
    ] * (curr_distance - df.iloc[last_ref_distance_index]["distance"])

    curr_distance += 250
    interpolated_heights.append(val)

# %%
plt.figure()
plt.scatter(np.array(distances) / 1000, interpolated_heights)
plt.xlabel("Distance (km)")
plt.ylabel("Height (m)")
plt.show()

# %%
# save distances and interpolated_heights to csv

# Save as tab-separated values (TSV)
np.savetxt(
    "../data/interpolated_heights.txt",
    np.column_stack([distances, interpolated_heights]),
    delimiter="\t",
    header="distance\theight",
    comments="",
)

# %%
# Load the results from Julia and plot
import pandas as pd
import matplotlib.pyplot as plt

results = pd.read_csv("../data/solution_results.txt", sep="\t")

plt.figure(figsize=(15, 5))
plt.plot(
    results["distance"], interpolated_heights - results["R"], label="R", marker="o"
)

# Mark where x == 1
selected = results[results["x"] == 1]

```

```
selected_heights = np.array(interpolated_heights)[results["x"] == 1]
plt.scatter(
    selected["distance"],
    selected_heights,
    color="red",
    label="a bomb location",
    zorder=5,
)
plt.scatter(distances, interpolated_heights)
plt.xlabel("Distance (km)")
plt.ylabel("Elevation (m)")
# plt.title("R values with x == 1 highlighted")
plt.legend()
plt.grid(True)
plt.show()

# %%
# count how many bombs were placed
num_bombs = results["x"].sum()
print(f"Number of bombs placed: {num_bombs}")

# %%
# Load the results from Julia and plot
import pandas as pd
import matplotlib.pyplot as plt


results = pd.read_csv("../data/solution_results.txt", sep="\t")
num_bombs = results["x"].sum()
depth = interpolated_heights - results["R"] # R is removed dirt
selected = results[results["x"] == 1]
selected_heights = np.array(depth)[results["x"] == 1]

plt.figure(figsize=(15, 5))

plt.plot(results["distance"], depth, label="depth", marker="o")

for x in selected["distance"]:
    plt.axvline(x, color="red", alpha=0.2, linestyle="--", label="bombs")

# Avoid duplicate legend entries
handles, labels = plt.gca().get_legend_handles_labels()
by_label = dict(zip(labels, handles))
plt.legend(by_label.values(), by_label.keys())

plt.grid(True)

plt.xlabel("Distance (km)")
plt.ylabel("Depth (m)")
plt.show()

num_bombs = results["x"].sum()
print(f"Number of bombs placed: {num_bombs}")

# %%
plt.figure(figsize=(15, 5))

results = pd.read_csv("../data/smooth_solution_results.txt", sep="\t")
num_bombs = results["x"].sum()
```

```
depth = interpolated_heights - results["R"] # R is removed dirt

# Mark where x == 1
selected = results[results["x"] == 1]

selected_heights = np.array(depth)[results["x"] == 1]
for x in selected["distance"]:
    plt.axvline(x, color="red", alpha=0.2, linestyle="--", label="bombs")

plt.plot(results["distance"], depth, label="depth", marker="o")
# Avoid duplicate legend entries
handles, labels = plt.gca().get_legend_handles_labels()
by_label = dict(zip(labels, handles))
# plt.scatter(distances, interpolated_heights)

plt.legend(by_label.values(), by_label.keys())

plt.xlabel("Distance (km)")
plt.ylabel("Depth (m)")
plt.grid(True)
plt.show()

print(f"Number of bombs placed: {num_bombs}")

# %%
# Load the results from Julia and plot
import pandas as pd
import matplotlib.pyplot as plt


plt.figure(figsize=(15, 5))
results = pd.read_csv("../data/smooth_solution_results.txt", sep="\t")
depth = interpolated_heights - results["R"] # R is removed dirt
plt.plot(results["distance"], depth, label="depth (smooth)", marker="o", alpha=0.5)

# Mark where x == 1
selected = results[results["x"] == 1]

selected_heights = np.array(depth)[results["x"] == 1]
plt.scatter(
    selected["distance"],
    selected_heights,
    color="red",
    label="bombs (smooth)",
    zorder=5,
    # set opacity to 0.5
    alpha=0.33,
)
# plt.scatter(distances, interpolated_heights)
results = pd.read_csv("../data/solution_results.txt", sep="\t")
depth = interpolated_heights - results["R"] # R is removed dirt
plt.plot(results["distance"], depth, label="depth (min nukes)", marker="o", alpha=0.5)
selected = results[results["x"] == 1]
selected_heights = np.array(depth)[results["x"] == 1]
plt.scatter(
    selected["distance"],
    selected_heights,
    color="blue",
    label="bombs (min nukes)",
```

```
    zorder=5,
    # set opacity to 0.5
    alpha=0.33,
)

results = pd.read_csv("../data/smooth_no_adj_solution_results.txt", sep="\t")
depth = interpolated_heights - results["R"] # R is removed dirt
plt.plot(
    results["distance"], depth, label="depth (smooth no adj)", marker="o", alpha=0.33
)
selected = results[results["x"] == 1]
selected_heights = np.array(depth)[results["x"] == 1]
plt.scatter(
    selected["distance"],
    selected_heights,
    color="yellow",
    label="bombs (smooth no adj)",
    zorder=5,
    # set opacity to 0.5
    alpha=0.33,
)


plt.xlabel("Index")
plt.ylabel("R")
plt.title("R values with x == 1 highlighted")
plt.legend()
plt.grid(True)
plt.show()

# %%
plt.figure(figsize=(15, 5))

results = pd.read_csv("../data/smooth_no_adj_solution_results.txt", sep="\t")
num_bombs = results["x"].sum()
depth = interpolated_heights - results["R"] # R is removed dirt

# Mark where x == 1
selected = results[results["x"] == 1]

selected_heights = np.array(depth)[results["x"] == 1]
for x in selected["distance"]:
    plt.axvline(x, color="red", alpha=0.2, linestyle="--", label="bombs")

plt.plot(results["distance"], depth, label="depth", marker="o")
# Avoid duplicate legend entries
handles, labels = plt.gca().get_legend_handles_labels()
by_label = dict(zip(labels, handles))
# plt.scatter(distances, interpolated_heights)

plt.legend(by_label.values(), by_label.keys())

plt.xlabel("Distance (km)")
plt.ylabel("Depth (m)")
plt.grid(True)
plt.show()

print(f"Number of bombs placed: {num_bombs}")

```

```
# %% [markdown]
# ### Dial a yield - is it worth it?

# %%
# below is only for the first 250 entries, otherwise solver runs >10 min.

# %%
# Load the results from Julia and plot
import pandas as pd
import matplotlib.pyplot as plt

entries = 250


# plt.figure(figsize=(15, 5))
# subplots
fig, axs = plt.subplots(2, 1, figsize=(20, 5))

results = pd.read_csv("../data/smooth_no_adj_solution_results.txt", sep="\t")
results = results[:entries]
ih = np.array(interpolated_heights)[:entries]
depth = ih - results["R"] # R is removed dirt
axs[0].plot(
    results["distance"], depth, label="depth (smooth no adj)", marker="o", alpha=0.5
)
selected = results[results["x"] == 1]
selected_heights = np.array(depth)[results["x"] == 1]
axs[0].scatter(
    selected["distance"],
    selected_heights,
    color="yellow",
    label="bombs (smooth no adj)",
    zorder=5,
    # set opacity to 0.5
    alpha=0.5,
)

results = pd.read_csv(
    "../data/smooth_no_adj_dial_k2_solution_results_250.txt", sep="\t"
)
# results = results[:entries]
ih = np.array(interpolated_heights)[:entries]
depth = ih - results["R"] # R is removed dirt
axs[1].plot(
    results["distance"], depth, label="depth (smooth no adj k2)", marker="o", alpha=0.5
)
selected = results[(results["x1"] == 1) | (results["x2"] == 1)]
selected_heights = np.array(depth)[(results["x1"] == 1) | (results["x2"] == 1)]
axs[1].scatter(
    selected["distance"],
    selected_heights,
    color="cyan",
    label="bombs (smooth no adj k2)",
    zorder=5,
    # set opacity to 0.5
    alpha=0.5,
)


```

```
plt.xlabel("Index")
plt.ylabel("R")
plt.title("R values with x == 1 highlighted")
plt.legend()
plt.grid(True)
plt.show()

# %%
plt.figure(figsize=(15, 5))

results = pd.read_csv("../data/smooth_no_adj_dial_solution_results.txt", sep="\t")
num_bombs = results["x1"].sum() + results["x2"].sum() + results["x3"].sum()
depth = interpolated_heights[:250] - results["R"] # R is removed dirt

# Mark where x == 1
selected = results[results["x1"] == 1]
selected_heights = np.array(depth)[results["x1"] == 1]
for x in selected["distance"]:
    plt.axvline(x, color="orange", alpha=0.5, linestyle="--", label="bombs")
selected = results[results["x2"] == 1]
selected_heights = np.array(depth)[results["x2"] == 1]
for x in selected["distance"]:
    plt.axvline(x, color="black", alpha=0.5, linestyle="--", label="bombs")
selected = results[results["x3"] == 1]
selected_heights = np.array(depth)[results["x3"] == 1]
for x in selected["distance"]:
    plt.axvline(x, color="blue", alpha=0.3, linestyle="--", label="bombs")

plt.plot(results["distance"], depth, label="depth", marker="o")
# Avoid duplicate legend entries
handles, labels = plt.gca().get_legend_handles_labels()
by_label = dict(zip(labels, handles))
# plt.scatter(distances, interpolated_heights)

plt.legend(by_label.values(), by_label.keys())

plt.xlabel("Distance (km)")
plt.ylabel("Depth (m)")
plt.grid(True)
plt.show()

print(f"Number of bombs placed: {num_bombs}")

# %%
plt.figure(figsize=(15, 5))

results = pd.read_csv("../data/smooth_no_adj_solution_results_250.txt", sep="\t")
num_bombs = results["x"].sum()
depth = interpolated_heights[:250] - results["R"] # R is removed dirt

# Mark where x == 1
selected = results[results["x"] == 1]
selected_heights = np.array(depth)[results["x"] == 1]
for x in selected["distance"]:
    plt.axvline(x, color="orange", alpha=0.5, linestyle="--", label="bombs")

plt.plot(results["distance"], depth, label="depth", marker="o")
# Avoid duplicate legend entries
handles, labels = plt.gca().get_legend_handles_labels()
```

```
by_label = dict(zip(labels, handles))
# plt.scatter(distances, interpolated_heights)

plt.legend(by_label.values(), by_label.keys())

plt.xlabel("Distance (km)")
plt.ylabel("Depth (m)")
plt.grid(True)
plt.show()

print(f"Number of bombs placed: {num_bombs}")
```